数据结构

```
|
├── 本质
|   ├── 数据怎么存
|   ├── 数据之间怎么组织
|   └── 怎么操作数据
|
├── 数组
|   ├── 存储：连续内存
|   ├── 类比：一排编号固定的存储柜
|   ├── 访问：下标
|   ├── 地址：数组元素地址 = 首地址 + 下标 × 元素大小
|   ├── 特点：查得快，插删慢
|   ├── 时间复杂度
|   |   ├── 下标查找：O(1)
|   |   ├── 尾插/尾删：O(1)
|   |   ├── 按值查找：O(n)
|   |   └── 中间插/删：O(n)
|   └── 适合：读多写少，随机访问
|
├── 链表
|   ├── 存储：非连续存储，通过指针连接（节点 + 指针 ，不连续）
|   ├── 类比：一群手拉手的人
|   ├── 特点：查找慢，已知位置插删快
|   ├── 时间复杂度
|   |   ├── 按位置查找：O(n)
|   |   ├── 按值查找：O(n)
|   |   ├── 已知节点插入：O(1)
|   |   ├── 已知节点删除：O(1)
|   |   └── 先查再删：O(n)
|   ├── 单链表：只有 next
|   ├── 双向链表：prev + next
|   └── 适合：频繁插入删除
|
├── 数组 vs 链表
|   ├── 问题1：有没有定位？
|   |   ├── 没有：按值查找都 O(n)
|   |   └── 有：数组下标 O(1)，链表按位置 O(n)
|   |
|   ├── 问题2：有没有移动？
|   |   ├── 数组中间插删：移动 → O(n)
|   |   └── 链表中间插删：改指针 → O(1)
|   |
|   └── 结论
|       ├── 数组：按下标 O(1)，中间插删 O(n)
|       └── 链表：按位置查 O(n)，已知节点插删 O(1)
```

```
│
├── 栈
│   ├── 存储：一种线性逻辑结构，可以是数组也可以是链表。特色是：后进先出 LIFO
│   ├── 类比：坐电梯，最后进去的人，最先出来
│   ├── 操作
│   │   ├── push：入栈
│   │   ├── pop：出栈，删除栈顶元素并返回它
│   │   └── peek：查看栈顶，不增不减
│   ├── 特点
│   │   ├── 只能在栈顶操作
│   │   └── 入栈/出栈/查看栈顶：O(1)
│   ├── 典型应用
│   │   ├── 函数调用：调用一个函数 → 入栈，函数结束 → 出栈
│   │   ├── 括号匹配：遇到左括号入栈，遇到右括号出栈
│   │   ├── 表达式求值
│   │   ├── 浏览器后退
│   │   └── DFS：深度优先搜索，先走到底，再回溯（不撞南墙不回头），常见“递归”
│   └── 记忆
│       ├── 栈 → 后进先出
│       ├── 栈 → 最后进去，最先出来
│       └── DFS → 深度 → 栈
│
├── 队列
│   ├── 存储：一种线性逻辑结构，可以是数组也可以是链表。特色是：先进先出 FIFO
│   ├── 类比：排队买奶茶，先来的人，先出去
│   ├── 操作
│   │   ├── enqueue：入队
│   │   ├── dequeue：出队，删除队头元素并返回它
│   │   └── front：查看队头，不增不减
│   ├── 特点
│   │   ├── 队尾入，队头出
│   │   └── 入队/出队/查看队头：O(1)
│   ├── 循环队列
│   │   ├── 解决：普通数组队列空间不能重复利用的问题
│   │   ├── 队空：front == rear
│   │   ├── 队满：(rear + 1) % QueueSize == front
│   │   ├── 关键：牺牲一个存储单元，用来区分队空和队满
│   │   └── 特点：rear 到数组末尾后，可以绕回开头继续使用
│   ├── 典型应用
│   │   ├── 打印排队
│   │   ├── 消息队列
│   │   ├── 任务队列
│   │   └── BFS：广度优先搜索
│   └── 记忆
│       ├── 队列 → 先进先出
│       ├── 队列 → 先来先走
│       ├── BFS → 广度 → 队列
│       └── BFS → 像剥洋葱，一层一层来
```

```
│
├── 树与二叉树
│   ├── 树
│   │   ├── 定义：从上往下分叉的层级结构
│   │   ├── 根节点：最顶端
│   │   ├── 叶节点：没有子节点
│   │   ├── 节点的度：有几个子节点
│   │   ├── 联想：公司组织架构图
│   │   └── 树的度：所有节点度的最大值 ★
│   ├── 二叉树
│   │   ├── 定义：每个节点最多 2 个子节点，分左右
│   │   ├── 注意：最多 2 个，不是必须 2 个 ★
│   │   ├── 满二叉树
│   │   │   └── 每个非叶节点都有 2 个子节点，通俗讲"儿女双全"
│   │   ├── 完全二叉树
│   │   │   ├── 除最后一层外，上面每层都填满
│   │   │   └── 最后一层从左往右连续排列，不能空缺
│   │   └── 关系
│   │       ├── 满二叉树 → 一定是完全二叉树 ✓
│   │       └── 完全二叉树 → 不一定是满二叉树 ✗
│   ├── 二叉排序树 BST ☆
│   │   ├── 别称：二叉搜索树 / 二叉查找树
│   │   ├── 本质：一种特殊的二叉树结构（不是遍历方式！）
│   │   ├── 核心性质：左小右大 ★
│   │   │   ├── 左子树所有节点值 < 根节点值
│   │   │   └── 右子树所有节点值 > 根节点值
│   │   ├── 与中序遍历的关系 ★★★
│   │   │   └── BST 做中序遍历 → 升序有序序列（非常好考）
│   │   ├── 时间复杂度
│   │   │   ├── 平均：查找 / 插入 / 删除 → O(logn)
│   │   │   └── 最坏（退化成链表）：O(n)
│   │   ├── 查找
│   │   │   └── 比根小 → 走左，比根大 → 走右，等于 → 找到
│   │   ├── 插入
│   │   │   └── 按查找路径走，走到空的地方就是插入位置
│   │   └── 删除（三种情况）
│   │       ├── 叶子节点 → 直接删
│   │       ├── 只有一个孩子 → 孩子顶上来
│   │       └── 有两个孩子 → 用"右子树最小值"或"左子树最大值"替换
│   └── 二叉树遍历
│       ├── 核心思路：拆，递归拆小树
│       ├── 前序
│       │   ├── 顺序：根 → 左 → 右
│       │   ├── 口诀：根左右
│       │   └── 联想记忆：
│       │       先访问根（分岔处），再往左走到底；
│       │       左边逛完，再回到分岔处去右边。
│       │       每个小树都遵循"根-左-右"，循环往复。
```

```
│       ├── 中序
│       │   ├── 顺序：左 → 根 → 右
│       │   ├── 口诀：左根右
│       │   └── 联想记忆：
│       │       先走到最左边（左子树）；
│       │       左边逛完，回到分岔处（根），再去右边（右子树）。
│       │       每个小树都遵循"左-根-右"，循环往复。
│       └── 后序
│           ├── 顺序：左 → 右 → 根
│           ├── 口诀：左右根
│           └── 联想记忆：
│               左边逛完，右边逛完，回到分岔处（根）。
│               循环往复往上走。
│
└── 图
    ├── 基本概念
    │   ├── 定义：G = V + E（顶点 + 边）
    │   ├── 本质：更自由的网络
    │   ├── 与树的关系：树 = 有特殊限制的图
    │   ├── 度
    │   │   ├── 无向图：度 = 连接该顶点的边数
    │   │   └── 有向图：入度 + 出度 = 该顶点的度
    │   ├── 入度（有向图）
    │   │   ├── 含义：有多少条边"指向"这个顶点
    │   │   ├── 例子：微博上，入度 = 粉丝数（多少人关注了我）
    │   │   └── 记忆：箭"进来"的数量
    │   └── 出度（有向图）
    │       ├── 含义：这个顶点"指出去"多少条边
    │       ├── 例子：微博上，出度 = 我关注了多少人
    │       └── 记忆：箭"出去"的数量
    ├── 图的分类
    │   ├── 按方向
    │   │   ├── 无向图
    │   │   │   ├── A ↔ B（互相可达）
    │   │   │   ├── 例子：双向通行道、互相认识
    │   │   │   └── 矩阵特点：主对角线对称
    │   │   └── 有向图
    │   │       ├── A → B，B 不一定 → A
    │   │       ├── 例子：微博关注、单行道
    │   │       └── 矩阵特点：不一定对称
    │   └── 按权值
    │       └── 带权图
    │           ├── 边上有数值（权）
    │           └── 例子：距离、时间、费用、网络延迟、运输成本
    ├── 存储方式
    │   ├── 邻接矩阵
    │   │   ├── 本质：二维数组（关系表）
    │   │   ├── 判断：A行B列 = 1 → A到B有边
```

```
│   │   ├── 优点：查边快，O(1)
│   │   ├── 缺点：稀疏图浪费空间
│   │   └── 对称性：无向图对称，有向图不一定
│   └── 邻接表
│       ├── 本质：数组 + 链表
│       ├── 思想：只存"我和谁连接"
│       ├── 优点：稀疏图省空间
│       └── 适用：边少、节点多、连接少
├── 对比总结
│   ├── 查边速度：矩阵 O(1) > 邻接表
│   ├── 空间：稀疏图用邻接表，稠密图用矩阵
│   ├── 矩阵 → 二维数组
│   └── 邻接表 → 数组 + 链表
├── 核心算法
│   ├── 图的遍历 ☆
│   │   ├── DFS（深度优先搜索）
│   │   │   ├── 数据结构：栈
│   │   │   ├── 核心思想：一条路走到黑
│   │   │   ├── 过程：从起点出发，碰到能走的就往下走，走到底再回溯换路
│   │   │   ├── 实现：递归 / 显式栈
│   │   │   └── 记忆：深度 → 栈 → 一条路走到黑
│   │   └── BFS（广度优先搜索）
│   │       ├── 数据结构：队列
│   │       ├── 核心思想：剥洋葱，一层一层来
│   │       ├── 过程：从起点出发，先走完所有相邻节点，再走下一层
│   │       ├── 实现：队列
│   │       └── 记忆：广度 → 队列 → 剥洋葱
│   ├── 最短路径 vs 最小生成树 ☆☆☆
│   │   ├── 最短路径：只管"走"
│   │   │   ├── 关心：两个点之间怎么走近
│   │   │   ├── 类比：开车从家到公司，走哪条路最近
│   │   │   ├── 产物：一条条路径
│   │   │   └── 代表算法：Dijkstra / Floyd
│   │   ├── 最小生成树：只管"连"
│   │   │   ├── 关心：整张图怎么用最少成本连通所有点
│   │   │   ├── 类比：给全村拉水管，怎么铺管子总长度最短
│   │   │   ├── 产物：一棵树（n 个点恰好 n-1 条边，无环）
│   │   │   └── 代表算法：Prim / Kruskal
│   │   └── 关键误区 ⚠
│   │       └── 最小生成树上两点的路径 ≠ 原图的最短路径
│   │           （MST 追求全局总代价最小，不保证每对点对点都最近）
│   ├── 最短路径 ☆
│   │   ├── Dijkstra（迪杰斯特拉）
│   │   │   ├── 类型：单源最短路径 / 找最近，再更新。
│   │   │   ├── 含义：从一个起点出发，到其他所有点的最短路
│   │   │   ├── 适用：边权非负（不能有负权边）
│   │   │   ├── 思想：贪心，每次选最近的点向外扩张
│   │   │   ├── 时间复杂度：O(n²) 或 O((n+m)logn)
```

```
    │   │   │   └── 记忆：单源 → 单起点 → 一个地方出发找最短路
    │   │   └── Floyd
    │   │       ├── 类型：任意两点最短路径  /   原路和绕中间点，谁短留谁
    │   │       ├── 含义：图中任意两个点之间的最短路
    │   │       ├── 适用：可以处理负权边（但不能有负权环）
    │   │       ├── 思想：动态规划，"经过某个中间点会不会更短"
    │   │       ├── 时间复杂度：O(n³)
    │   │       └── 记忆：多源 → 全部互找 → 任意两点找最短路
    │   └── 最小生成树 ⭐
    │       ├── 前提：连通无向带权图
    │       ├── 目标：选 n-1 条边，连通所有点，总权值最小
    │       ├── Prim
    │       │   ├── 思想：从一个点向外扩张    /  烟花向外炸，从已连接区域往外找最便宜
    │       │   ├── 过程：维护一个已连通的点集，每次找从集合内到集合外的最小边，把对端点纳入集合
    │       │   ├── 时间复杂度：O(n²) 或 O((n+m)logn)
    │       │   └── 记忆：从点出发 → 向外扩张
    │       └── Kruskal
    │           ├── 思想：最小边优先，不能成环
    │           ├── 过程：把所有边按权值从小到大排序，依次选边，选之前判断加进去会不会形成环（并查集）
    │           ├── 时间复杂度：O(mlogm)（排序）
    │           └── 记忆：从边出发 → 贪最小 → 并查集判环
    └── 秒选口诀
        ├── 快速判断两点是否有边 → 邻接矩阵
        ├── 稀疏图 / 边少节点多 → 邻接表
        ├── 主对角线对称 → 无向图
        ├── 数组 + 链表 → 邻接表
        ├── 二维数组 → 邻接矩阵
        ├── 微博关注 / 单行道 → 有向图
        ├── 互相认识 / 双向道 → 无向图
        ├── DFS → 栈 → 一条路走到黑
        ├── BFS → 队列 → 剥洋葱
        ├── 单源最短路径 → Dijkstra
        ├── 任意两点最短路径 → Floyd
        ├── 最小生成树：从点扩张 → Prim
        └── 最小生成树：从边贪心 → Kruskal
```

> 数组看“下标”，链表看“节点”，栈看“最后”，队列看“最先”。